```
#include <iostream>
#include <iomanip>
#include <array>
#include <string>
using namespace std;

// All money values are stored in integer paise.
constexpr long long MIN_BALANCE_PAISE = 5000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;

// Four zones support future network expansion.
enum class Zone {
    Inner = 0,
    Middle = 1,
    Outer = 2,
    Express = 3
};

constexpr array<long long, 4> BASE_FARE_PAISE = {
    1000, 2000, 3000, 4000
};

class FareCalculator {
public:
    static long long calculateFare(
        Zone zone, bool peak, bool weekend, bool monthlyPass) {
```

```
        if (monthlyPass) {
            return 0;
        }

        int index = static_cast<int>(zone);

        if (index < 0 || index >=
            static_cast<int>(BASE_FARE_PAISE.size())) {
            return -1;
        }

        long long fare = BASE_FARE_PAISE[index];

        if (peak) {
            fare = fare * (100 + PEAK_SURCHARGE_PERCENT) / 100;
        }

        if (weekend) {
            fare = fare * (100 - WEEKEND_DISCOUNT_PERCENT) / 100;
        }

        return fare;
    }
};

class SmartCard {
private:
    long long balancePaise;
```

```
public:
    explicit SmartCard(long long initialBalance)
        : balancePaise(initialBalance >= 0 ? initialBalance : 0) {}

    long long getBalance() const {
        return balancePaise;
    }

    bool canTravel(long long fare) const {
        return fare >= 0 &&
               balancePaise >= fare &&
               balancePaise - fare >= MIN_BALANCE_PAISE;
    }

    bool deduct(long long fare) {
        if (!canTravel(fare)) {
            return false;
        }

        balancePaise -= fare;
        return true;
    }
};

void printMoney(long long paise) {
    cout << "Rs. " << paise / 100 << "."
         << setw(2) << setfill('0') << paise % 100
```

```
        << setfill(' ');
}

int main() {
    cout << "NAMMA METRO - FINAL DESIGN\n\n";

    SmartCard card(10000);

    struct Journey {
        string name;
        Zone zone;
        bool peak;
        bool weekend;
        bool monthlyPass;
    };

    Journey journeys[] = {
        {"Inner zone weekday", Zone::Inner, false, false, false},
        {"Outer zone peak hour", Zone::Outer, true, false, false},
        {"Express zone weekend", Zone::Express, false, true, false},
        {"Monthly pass journey", Zone::Outer, true, false, true}
    };

    for (const Journey& journey : journeys) {
        long long fare = FareCalculator::calculateFare(
            journey.zone,
            journey.peak,
            journey.weekend,
```

```
        journey.monthlyPass
    );

    cout << journey.name << " | Fare: ";

    if (fare < 0) {
        cout << "Invalid zone\n";
        continue;
    }

    printMoney(fare);

    if (card.deduct(fare)) {
        cout << " | Accepted | Balance: ";
        printMoney(card.getBalance());
        cout << '\n';
    } else {
        cout << " | Denied: minimum balance rule | Balance: ";
        printMoney(card.getBalance());
        cout << '\n';
    }
}

cout << "\nFinal balance: ";
printMoney(card.getBalance());
cout << '\n';

return 0;
```

```
}
```